Shivani.S-192471030

Q66. Intelligent Document Search Engine Optimization (Indexing + Binary Search)

A search engine must retrieve relevant documents from massive datasets quickly. Analyze how indexing and binary search techniques can improve retrieval speed. Identify constraints such as large data volume and query complexity. Design an optimized search system. Discuss scalability for web-scale data. Evaluate performance metrics. Interpret results in user search Efficiency.

Ans:

```
# Q66. Intelligent Document Search Engine Optimization
# Using Indexing + Binary Search

from bisect import bisect_left
def tokenize(text):
    words = text.lower().split()
    cleaned_words = []

    for word in words:
        cleaned = ""
        for ch in word:
            if ch.isalnum():
                cleaned += ch

        if cleaned:
            cleaned_words.append(cleaned)

    return cleaned_words

def build_index(documents):
    inverted_index = {}

    for doc_id, text in documents.items():
        words = tokenize(text)

        for word in words:
```

```
            if word not in inverted_index:
                inverted_index[word] = []
            if doc_id not in inverted_index[word]:
                inverted_index[word].append(doc_id)

    for word in inverted_index:
        inverted_index[word].sort()

    sorted_terms = sorted(inverted_index.keys())

    return inverted_index, sorted_terms


def binary_search_term(sorted_terms, target):
    index = bisect_left(sorted_terms, target)

    if index < len(sorted_terms) and sorted_terms[index] == target:
        return index

    return -1


def search_documents(query, documents, inverted_index, sorted_terms):
    query_words = tokenize(query)
    result_count = {}

    for word in query_words:
        position = binary_search_term(sorted_terms, word)

        if position != -1:
            matched_word = sorted_terms[position]
            doc_list = inverted_index[matched_word]

            for doc_id in doc_list:
                result_count[doc_id] = result_count.get(doc_id, 0) + 1

    ranked_results = sorted(result_count.items(), key=lambda x: x[1], reverse=True)

    return ranked_results
```

```
# Sample Documents
documents = {
    1: "Data structures and algorithms are important for search engines",
    2: "Binary search improves search speed in sorted data",
    3: "Indexing helps retrieve documents quickly from large datasets",
    4: "Search engines use indexing and ranking techniques"
}

# Build Index
inverted_index, sorted_terms = build_index(documents)

# Query
query = "search indexing"

results = search_documents(query, documents, inverted_index, sorted_terms)

print("Search Query:", query)
print("\nSearch Results:")

if not results:
    print("No documents found")
else:
    for doc_id, score in results:
        print("Document ID:", doc_id)
        print("Score:", score)
        print("Text:", documents[doc_id])
        print()
```

Q67. Smart Energy Consumption Prediction (Dynamic Programming + Forecasting)

An energy provider must predict future consumption patterns to optimize generation and distribution. Analyze how dynamic programming and forecasting techniques can be applied. Identify constraints such as seasonal variations and data uncertainty. Design a predictive model. Discuss scalability for large regions. Evaluate computational efficiency. Interpret results in energy management systems.

Ans:

```
# Q67. Smart Energy Consumption Prediction
# Using Forecasting + Dynamic Programming

def forecast_consumption(history, alpha):
 forecast = [history[0]]

 for i in range(1, len(history)):
 predicted = alpha * history[i - 1] + (1 - alpha) * forecast[i - 1]
 forecast.append(predicted)

 next_forecast = alpha * history[-1] + (1 - alpha) * forecast[-1]

 return next_forecast


def optimize_generation(demand, generators):
 max_capacity = demand + max(gen["capacity"] for gen in generators)

 dp = [float("inf")] * (max_capacity + 1)
 choice = [-1] * (max_capacity + 1)

 dp[0] = 0

 for energy in range(1, max_capacity + 1):
 for i in range(len(generators)):
 capacity = generators[i]["capacity"]
```

```
cost = generators[i]["cost"]

if energy - capacity >= 0:
if dp[energy - capacity] + cost < dp[energy]:
dp[energy] = dp[energy - capacity] + cost
choice[energy] = i
else:
if cost < dp[energy]:
dp[energy] = cost
choice[energy] = i

best_energy = demand
best_cost = dp[demand]

for energy in range(demand, max_capacity + 1):
if dp[energy] < best_cost:
best_cost = dp[energy]
best_energy = energy

selected_generators = []
current = best_energy

while current > 0 and choice[current] != -1:
gen_index = choice[current]
selected_generators.append(generators[gen_index]["name"])
current -= generators[gen_index]["capacity"]
```

```
 return best_energy, best_cost, selected_generators



# Historical energy consumption data
# Example: daily consumption in megawatt-hours
history = [120, 130, 125, 140, 150, 160, 155]

alpha = 0.6

predicted_demand = forecast_consumption(history, alpha)
predicted_demand = int(round(predicted_demand))

generators = [
 {"name": "Solar Plant", "capacity": 50, "cost": 300},
 {"name": "Wind Plant", "capacity": 70, "cost": 400},
 {"name": "Thermal Plant", "capacity": 100, "cost": 700}
]

energy, cost, selected = optimize_generation(predicted_demand, generators)

print("Predicted Energy Demand:", predicted_demand)
print("Generated Energy:", energy)
print("Minimum Generation Cost:", cost)
print("Selected Generators:")

for generator in selected:
 print(generator)
```